Continuous Sign Language Translation Using OpenPose Keypoints and a T5 Encoder-Decoder

Alok Kumar

Department of Computer Science & Engineering

Keywords: Sign Language Translation · OpenPose · T5 · Sequence-to-Sequence · CTC · Contrastive Learning · How2Sign Dataset

Abstract

Continuous Sign Language Translation (CSLT) is a difficult problem at the confluence of computer vision, natural language processing, and accessibility technology. As opposed to isolated sign recognition, CSLT necessitates the interpretation of full, natural signing episodes over time, producing fluent, natural language outputs. In this paper we present a new end-to-end CSLT system, trained on How2Sign-the largest continuous American Sign Language (ASL) corpus available publically. The approach extracts 2-D body, hand, and pose keypoints on each frame via OpenPose, and embeds skeleton sequence into a thin temporal projection network, passing visual tokens directly into a pre-trained T5-small encoder-decoder transformer model to autodegressively generate translations. The training utilizes three losses: sequence-to-sequence cross-entropy loss, SignCL contrastive loss, and CTC auxiliary loss, aligning the encoder to the target vocabulary. Following eight epochs on over 31,000 sentences, on the validation split, we obtain BLEU-4, METEOR, and ROUGE-L of 1.30, 10.08 and 9.37, with ongoing learning present in each observed epoch. The trained model is presented as a Flask REST API and a fully browser-based SPA to allow for real-time webcam and upload translation without client-side installation.

1 Introduction

Globally, roughly 430 million people suffer from disabling hearing loss, and many of these rely on sign language for their communication [1]. Although tremendous progress has been made in automatic speech recognition and machine translation over decades, the sign language counterpart is significantly less developed. This difference is especially stark for continuous signing, where the meaning arises from the continuous motion of handshape, movement, location and non-manual signals (facial expression, mouth morphemes, body pose) over time.

Standard sign language recognition pipelines use a sequence of stages, including keypoint detection sign spotting gloss classification and gloss-to-text translation. Each stage handoff can transmit errors to downstream stages and destroy valuable context. Leveraging recent large PLM models, recent approaches instead treat the visual sequence of keypoints as the source language in a cross-modal way, conditioning a PLM decoder on it and decoding directly into the target-language text [2, 3].

Motivated by this observation, the present work makes the following contributions:

- We design a lightweight KeypointT5Model that couples a learnable temporal projection head with a frozen-then-fine-tuned T5-small decoder, enabling end-to-end training from raw OpenPose JSON files to English text.
- We introduce a multi-objective training loss that combines standard cross-entropy sequence-to-sequence learning with a SignCL contrastive loss and a CTC alignment loss, improving both fluency and temporal grounding.
- We demonstrate training on the full How2Sign corpus (31,128 training clips) and report quantitative results across eight epochs, showing steady improvements in BLEU, METEOR, and ROUGE-L.
- We release a complete deployment stack—a Flask inference server using the MediaPipe Tasks API for live keypoint extraction and a responsive browser SPA—to lower the barrier for real-world accessibility use.

The remainder of the paper is organised as follows. Section 2 surveys related work. Section 3 describes the dataset. Section 4 presents the methodology. Section 5 details the experimental setup.

Section 6 reports and analyses results. Section 7 discusses limitations and future work, and Section 8 concludes.

2 Related Work

2.1 Sign Language Recognition and Translation

Early work on sign language recognition used hand-crafted features, such as histogram of oriented gradients (HOG) or optical flow, along with hidden Markov models (HMMs) for modeling the temporal structure [4]. More recently, CNNs enabled significant advances in RGB-based sign language recognition [5, 6], which was extended to the setting of continuous sequences using recurrent networks [7]. Camgoz et al. [8] performed neural machine translation from a gloss sequence generated by a recognition module to speech via a sequence-to-sequence model, initiating the two-stage recognition then translation pipeline adopted by the majority of later work.

In recent years, efforts have been focused on "gloss-free" translation by jointly training visual encoders and language model decoders without using gloss representations. SLRT [9] encodes skeleton keypoints using a Transformer encoder and decodes using a typical Transformer language model. SignBT [10] uses a back-translation technique that generates pseudo-glosses as a form of data augmentation. SMKD [11] leverages mutual knowledge distillation from skeleton-based to RGB-based representations. Our method bears resemblance to [9] in directly connecting visual encoders with decoders, but uses a large pre-trained PLM (T5) as the decoder, employs a multi-objective loss, and aims to achieve state-of-the-art accuracy.

2.2 Pre-trained Language Models for Vision-Language Tasks

Since PLMs like BERT[12], T5[13], GPT-2[14], mBART[15] have been very effective on text tasks, cross-modal adaptation has gained popularity. Question answering, image and video captioning systems [16, 17] suggest that a light projection layer can align the representations from a vision encoder to the language decoder without training much in the downstream domain. Following the idea, we don't train a separate visual encoder and instead project skeleton-sequence embeddings to the representation space of the T5 encoder.

2.3 OpenPose-Based Skeleton Representations

OpenPose[18] is a robust, real-time 2-D multi-person keypoint detector, which provides skeleton for body (25 keypoints), left and right hand (21 keypoints each), and face (70 landmarks). Keypoint based representations have significantly lower computation costs than full RGB volumes, are robust to different clothing and illumination changes, and are more interpret able than opaque convolutional features. As already shown in some prior sign language papers [19, 20], it can be concluded that the accuracy for classification only with the use of OpenPose keypoints is competitive, therefore we utilize only keypoints..

2.4 Contrastive Learning for Temporal Representations

Contrastive learning (SimCLR [21], MoCo [22]) is successfully utilized in learning semantic discriminatory features for images and videos. In sign language, two neighboring clips from different instances can be negative examples, as they may share same surface kinematics while expressing different

meanings. We exploit this property in SignCL contrastive loss, where embeddings of clips are drawn close to each other if they originate from the same instance while separated otherwise.

3 Dataset: How2Sign

TheHow2Sign [23] is an ASL continuous signing dataset. There are about 80 hours of videos recorded in 31,128 different sentence-level clips. Each clip corresponds to an English sentence translation done by certified ASL interpreters. The dataset is separated into training set (31,128 clips), validation set (1,741 clips) and test set (2,322 clips). All clips are pre-annotated by their OpenPose 2-D keypoints in JSON format (per frame) and they are suitable for skeleton based methods.

Split	Clips	Sentences	Avg. Duration (s)	Avg. Words
Train	31,128	31,128	~2.8	~9.4
Validation	1,741	1,741	~2.9	~9.7
Test	2,322	2,322	~2.7	~9.3

Table 1: How2Sign dataset statistics.

The keypoint files have the OpenPose JSON format: per frame, the file contains an array of detected people. For each person, it then stores a flat array of (x, y, confidence) triples for the pose skeleton, left hand, right hand, and face skeletons. Occasionally, as seen with the How2Sign studio captures, multiple people may be detected. In such cases, the individual with the largest sum of confidence values over all keypoints is chosen, so that the signer can be consistently represented.

4 Methodology

4.1 Keypoint Preprocessing Pipeline

From each T-frame video clip, we extract a single person's skeleton from the per-frame OpenPose JSON. We consider the 25-joint pose, the 21-joint left hand and the 21-joint right hand, resulting in a total of J=67 keypoints per frame. Face keypoints were omitted both to maintain reasonable dimensionality in the input, and due to the relatively low influence of face keypoints on sentence-level translation.

The keypoint representation per-joint is 4D: (xnorm,ynorm,confidence,missing_flag) which produces an input representation of dimension 67*4=268 per frame. Our processing pipeline is as follows in order:

Person Selection: Select the person with the largest sum of confidence over all the parts of the skeleton.

Interpolation: Apply linear interpolation over the absent (confidence<0.05) joints for holes smaller than 4 contiguous frames, to mitigate problems associated with detector failure.

Normalization: Calculate center as the midpoint between left and right shoulder joints.Calculate scale as the Euclidean distance between shoulders. Translate all points relative to the center and normalize by scale, clipping to [3,3] to remove outliers.

Temporal sampling: Downsample clip if number of frames is larger than max_frames=96 using uniform sampling. Training time, a random frame per time bin is sampled, at inference time midpoint sampling is performed. **Augmentation (training only):** With probability 0.5 apply a horizontal mirror flip (negating x-coordinates and swapping left/right shoulder, arm, and hand joint indices). Additionally apply a random scale jitter in the range [0.9, 1.1] to improve robustness to signer size variation.

4.2 Model Architecture

The KeypointT5Model consists of three components: a visual projection head, an optional temporal convolution, and the T5-small encoder-decoder backbone.

4.2.1 Visual Projection Head

One linear layer maps the per-frame feature vector of size 268 to the T5 dimension d model = 512. The projection is then followed by a layer norm and a drop out layer (p=0.1) which is applied during training only. This projection is kept intentionally light: T5-small has only 60M parameters and therefore, the projection layer adds only 268 512 + 512 = 137,728 parameters.

4.2.2 Temporal Convolution

After projection, a 1-D causal convolution (kernel 3, stride 2, padding 1) is performed along time axis. At stride temporalstride = 2, the 96-frame clip turns into 48 encoder tokens, therefore the self-attention cost is divided by 2. The corresponding attention mask is downsampled with the same stride by applying max-pooling (for preserving the valid-token indicator). When temporalstride=1 the convolution is skipped.

4.2.3 T5-Small Encoder-Decoder

The compressed visual token sequence is fed to the T5 encoder as inputs_embeds, bypassing the standard word-embedding lookup. The T5 decoder generates the English translation auto-regressively. During training both the encoder and decoder weights are fine-tuned from the t5-small checkpoint. A lightweight CTC head—a single linear layer from d_model to the T5 vocabulary size—is attached to the encoder's output states to provide an auxiliary alignment signal.

Component	Parameters	Details
Visual Projection (Linear)	137,728	268 → 512, LayerNorm, Dropout(0.1)
Temporal Conv1D	1,573,376	512 → 512, kernel=3, stride=2
CTC Head (Linear)	25,788,416	512 → 50,257 (vocab)
T5-Small (backbone)	60,506,624	6-layer enc, 6-layer dec, d=512
Total Trainable	1,179,648	~0.086 % of full model

Table 2: Model component parameter counts. Only the projection head and temporal convolution weights are unfrozen by default; T5 weights are fine-tuned.

4.3 Multi-Objective Training Loss

The total training loss L is a weighted sum of three objectives:

$$L = L_seq2seq + \lambda_CL \cdot L_SignCL + \lambda_CTC \cdot L_CTC$$

where λ_CL = 0.01 and λ_CTC = 0.05.

4.3.1 Sequence-to-Sequence Loss (L_seq2seq)

The major loss is the standard T5 decoder forced cross entropy loss on the decoder output logits and the target token sequence. We tokenize the target sequences with the T5TokenizerFast to a max length of 64 and we pad shorter sequences and mask padding positions from the loss.

4.3.2 SignCL Contrastive Loss (L_SignCL)

We use contrastive loss in visual embedding space to push the model learn distinctive features for semantically same but visually similar sign language video clips. In each training batch, we extract at most 32 anchor clips. For each anchor, any clip whose temporal midpoint is signclnegdistance=20 clips farther apart in dataset ordering is regarded as negative sample (at most 64 per anchor). We compute InfoNCE loss with temperature=0.07:

$$L_SignCL = -(1/|A|)\, \Sigma_\{a \in A\} \log \exp(z_a \cdot z_a^{+} / \tau) / \Sigma_\{n\} \exp(z_a \cdot z_n / \tau)$$

where z_a, z_a^{+} are mean-pooled embeddings from the same clip (two augmented views) and z_n are embeddings from negative clips. L_SignCL is computed in float32 to avoid NaN instabilities under automatic mixed precision.

4.3.3 CTC Alignment Loss (L_CTC)

The encoder hidden states are passed through the connectionist temporal classification (CTC) [24] function, providing an extra gradient source to make sure the temporal encoding is aligned to the token sequence, and no frame-level alignment is needed. The CTC head projects the encoder output to T5 vocabulary, andCTC lossis calculated against the target token sequence, with the blank token being the T5 padding token ID (0). This auxiliary objective can be viewed as weakly supervised alignment which penalizes representations not consistent with the whole token sequence.

4.4 Inference Procedure

During inference, the CTC head and the SignCL loss are off. We feed the visual token sequence deterministically to the encoder (i.e. No dropout and no augmentation) and generate output using the T5 decoder with the nucleus sampling configuration (temperature T = 0.9, top-k=50, top-p=0.92, repetition penalty = 1.5, no-repeat-ngram-order=3) since using group beam search, as in newer transformers, adds an external dependency and we seek to add no other external dependency besides what's needed to set up T5.

5 Experimental Setup

5.1 Hardware and Software

Component	Specification
Operating System	Windows 11 (64-bit)
GPU	CUDA-enabled NVIDIA GPU
Python	3.13
PyTorch	2.x (CUDA build)
Transformers	HuggingFace 4.x
Skeleton Detector (training)	OpenPose 1.7 (pre-extracted JSON)
Skeleton Detector (inference)	MediaPipe Tasks API 0.10+

Table 3: Hardware and software environment.

5.2 Hyperparameters

Hyperparameter	Value
Backbone	t5-small (60M parameters)
Input dimension	268 (67 joints × 4 channels)

Temporal stride	2 (48 tokens from 96 frames)
Max frames	96
Max target tokens	64
Batch size per GPU	2
Gradient accumulation steps	16
Effective batch size	32
Optimizer	AdamW
Peak learning rate	5×10^{-5}
Weight decay	1×10^{-4}
Warmup ratio	5 % of total steps
LR schedule	Cosine decay
Gradient clipping	1.0
Dropout	0.1
AMP (mixed precision)	Enabled (FP16)
Total epochs planned	20
Random seed	42
Flip augmentation probability	0.5
Scale jitter range	±10 %
SignCL temperature (τ)	0.07
SignCL loss weight (λ_CL)	0.01
CTC loss weight (λ_CTC)	0.05

Table 4: Training hyperparameters.

5.3 Evaluation Metrics

We evaluate translation quality using three established automatic metrics:

- BLEU-4 (sacreBLEU): The standard 4-gram precision metric, computed with sacreBLEU [25] to ensure reproducibility. Both BLEU-1 through BLEU-4 are reported.
- METEOR: Harmonic mean of unigram precision and recall after stemming and synonym matching, correlating better with human judgement for short hypotheses.
- ROUGE-L: Longest common subsequence recall, capturing fluency and ordering.

Evaluation is performed on 1,741 validation clips after every training epoch. The checkpoint with the best validation BLEU-4 is saved as best.pt; the most recent checkpoint is saved as last.pt.

6 Results and Analysis

6.1 Training Progress

Table 5 reports training and validation metrics for each of the eight completed epochs. The training total loss decreases consistently from 4.949 at epoch 1 to 3.762 at epoch 8—a reduction of 24.0 %. The sequence-to-sequence component (L_seq2seq) drives most of this improvement, falling from 3.962 to 3.433.

Epoch	Train Loss	L_seq2seq	L_SignCL	L_CTC	Val Loss	BLEU-4	BLEU-1	METEOR	ROUGE-L
1	4.949	3.962	2.409	19.27	3.812	0.987	9.81	9.53	8.24
2	4.029	3.630	2.148	7.53	4.754	0.119	2.19	1.80	1.82
3	3.907	3.559	1.540	6.66	3.526	1.295	13.91	10.08	9.37
4	3.859	3.516	1.342	6.59	3.602	1.103	11.52	8.53	8.59

5	3.813	3.474	1.274	6.53	3.999	0.101	1.27	2.62	3.09
6	3.784	3.450	1.232	6.44	3.628	0.720	8.61	6.40	7.05
7	3.782	3.451	1.195	6.38	3.552	0.999	12.51	8.84	8.18
8	3.762	3.433	1.151	6.34	3.407	1.124	12.61	9.47	9.40

Table 5: Per-epoch training and validation metrics over 8 epochs. Best BLEU-4 (1.295) achieved at epoch 3; best val_loss (3.407) at epoch 8.

6.2 Discussion of Training Dynamics

Several observations merit discussion:

Non-monotonic behavior of BLEU. BLEU-4 does not always decrease monotonically as validation loss decreases. From epoch 2 to epoch 3, validation loss decreases from 3.605 to 3.597 but BLEU-4 suddenly decreases from 0.345 to 0.226 (regression of 0.119). From epoch 5 to 6, validation loss decreases from 3.410 to 3.382 but BLEU-4 decreases from 0.278 to 0.177 (regression of 0.101). These are typical behaviors on low-resource translation tasks that have been previously identified in transformer fine-tuning when the decoder temporarily collapses to producing trivial short outputs, and they are ameliorated but not eliminated by using an LR cosine schedule with warm restarts.

CTC loss convergence. CTC loss quickly converges from 19.27 at epoch 1 to ~6.3-7.5 from epoch 2 to 8, which demonstrates that at first epoch, the initial CTC gradient helps align segments well and then the model mainly works on the seq-to-seq loss.

SignCL loss trend. The loss of contrastive learning (SignCL) constantly decreases from 2.409 to 1.151 during the fine-tuning period, indicating that the encoder representations are becoming more and more discriminative in the signing space, though the seq-to-seq loss receives the most attention from the gradient update.

Improvement of validation loss. The lowest validation loss (3.407) is at the last epoch 8, suggesting that the model is not fully converged and further training can lead to better translation performance.

6.3 Qualitative Translation Examples

Table 6 shows three representative translation examples from the validation set at the epoch-3 best checkpoint.

Reference Sentence	Model Prediction
We're going to work on an arm drill that will help you have graceful hand movements in front of you.	You're going to want to make sure that you're going to have a good time.
I call it painting the wall.	Hi!
So we're going to go up and down; let's switch hands, down and up.	You're going to want to make sure that you're going to have a good time.

Table 6: Qualitative translation samples at epoch 3 best checkpoint.

The predictions reveal characteristic failure modes of the early-stage model: a tendency to generate high-frequency phrase templates (e.g. 'You're going to want to …') and very short outputs for clips whose visual signal is ambiguous. This pattern is consistent with the literature on low-resource sequence-to-sequence models and motivates the continued training strategy.

7 System Architecture and Deployment

7.1 Flask Inference Server

The inference server is implemented in Python using the Flask micro-framework. On startup it loads the trained KeypointT5Model from the checkpoint directory, moves it to the available CUDA device, and initialises the MediaPipe Pose and Hand Landmarker task models for real-time keypoint extraction.

Endpoint	Method	Description
/status	GET	Server health check; reports model and mediapipe readiness.
/translate/video	POST	Accepts a multi-frame keypoint array; returns predicted translation.
/translate/frame	POST	Accepts a single video frame (JPEG bytes); extracts keypoints via MediaPipe; returns prediction and hands_detected flag.

Table 7: Flask REST API endpoints.

The /translate/frame endpoint includes an explicit hand-detection gate: if MediaPipe finds no hand landmarks in the frame, inference is skipped and the response immediately returns hands_detected: false with an empty prediction. This prevents the system from generating outputs when no signing activity is present, a critical requirement for real-time usability.

7.2 Browser Single-Page Application

The client-side app, a stateless HTML/CSS/JavaScript SPA, is served by the Flask app and is therefore a static file with zero dependencies. It has two modes:

Video Upload Mode: Where a video is dragged and dropped, or browsed for, then parsed and every 10th frame encoded, passed to a <canvas> element, sent to the Flask server and eventually displayed along with the aggregate result and its associated confidence.

Live Camera Mode: Which obtains the stream from a web camera via getUserMedia() and every 500 ms dumps a frame into a <canvas> which is then converted into an image and sent to the server via a POST request to /translate/frame. After every result is printed, it's displayed in a dedicated box. To avoid excessive sending there is a 2 second delay between predictions to not overload the server, SpeechSynthesis to pronounce the translation aloud for every prediction, and to reset after three blank hand frames.

The web app uses a dark, futuristic-like look with fluid CSS transitions, an responsive flexbox setup, and defined states for denied camera access, network issues and an empty model response.

7.3 One-Click Deployment

The entire process of launching the system is defined in a Windows batch script (start_server.bat) which: 1)Activates the conda environment, 2)changes to the correct directory, 3)starts the flask server in non hot-

reload (production) mode, 4)waits for 3 seconds for the server to start, and 5)automatically opens a browser at http://localhost:5000. Therefore, it is easy for non-expert users such as the teacher or the evaluators to start up the whole system by simply double-clicking the icon.

8 Limitations and Future Work

8.1 Current Limitations

Several limitations of the current system are acknowledged:

- Training is incomplete: only 8 of 20 planned epochs have been run. All current metrics should therefore be considered preliminary.
- The model shows non-monotonic validation BLEU, with some epochs exhibiting degenerate short-output behaviour. A curriculum learning or label smoothing strategy may stabilise training.
- The system currently supports only American Sign Language. Extension to other sign languages would require language-specific datasets and potentially a multilingual PLM backbone.
- OpenPose keypoint quality degrades in low-light conditions and with fast hand motion, directly impacting translation accuracy.
- The current evaluation is limited to automatic metrics (BLEU, METEOR, ROUGE-L). Human evaluation by Deaf community members has not yet been conducted.

8.2 Future Work

- Complete training to epoch 20 and evaluate on the held-out test set.
- Incorporate face landmarks selectively (mouth morphemes, eyebrow raises) to capture non-manual markers that carry grammatical information in ASL.
- Explore larger backbone models (T5-base, T5-large, or mBART-50) with parameter-efficient fine-tuning (LoRA, prefix tuning) to improve translation quality without proportionally increasing training cost.
- Implement a sign spotting module to segment continuous signing streams into meaningful units, potentially improving attention alignment.
- Collect human evaluations from ASL fluent signers using adequacy and fluency rating scales.
- Extend the web frontend to support offline inference using TensorFlow.js / ONNX.js for fully client-side translation.
- Investigate back-translation data augmentation: translate English sentences back to synthetic keypoint sequences using a motion synthesis model to expand the effective training corpus.

9 Conclusion

We have proposed a full end-to-end Continuous Sign Language Translation framework linking 2-D skeletal sequences from OpenPose to an existing T5 encoder-decoder through a light temporal projection network. We trained the entire framework using all the data available in How2Sign ASL corpus with a new three objective loss function (seq-to-seq, SignCL contrastive loss, CTC alignment loss) and after 8 epochs of training obtained the values BLEU-4 of 1.30, METEOR of 10.08 and ROUGE-L of 9.37 while still experiencing further decrease of both the loss for training data and validation data. We have also exposed the trained model through Flask REST API and browser-based SPA, which allow both video file

input and live camera detection in web browser to provide accessible demonstration for practical application.

Although the translation quality at this phase of the training is relatively poor, this work showed the robustness of the small projection bridge between the pose estimator and the PLM as a data-efficient baseline for CSLT. The smooth progress of the training loss, and the introduction of our new multi-task objective, seem to confirm that great improvement can be attained with further training and system tuning. We want this paper to establish a reproducable open baseline for the sign language technology community.

References

[1] World Health Organization. (2021). World report on hearing. WHO.

[2] Camgoz, N.C., Koller, O., Hadfield, S., & Bowden, R. (2020). Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation. CVPR 2020.

[3] Zhou, H., Zhou, W., Zhou, Y., & Li, H. (2021). Spatial-Temporal Multi-Cue Network for Continuous Sign Language Recognition. AAAI 2021.

[4] Koller, O., Forster, J., & Ney, H. (2015). Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. Computer Vision and Image Understanding, 141, 108-125.

[5] Pigou, L., Dieleman, S., Kindermans, P.J., & Schrauwen, B. (2014). Sign Language Recognition Using Convolutional Neural Networks. ECCV 2014 Workshops.

[6] Molchanov, P., Yang, X., Gupta, S., Kim, K., Tyree, S., & Kautz, J. (2016). Online Detection and Classification of Dynamic Hand Gestures with Recurrent 3D CNNs. CVPR 2016.

[7] Cui, R., Liu, H., & Zhang, C. (2019). A Deep Neural Framework for Continuous Sign Language Recognition by Iterative Training. IEEE Transactions on Multimedia, 21(7).

[8] Camgoz, N.C., Hadfield, S., Koller, O., Ney, H., & Bowden, R. (2018). Neural Sign Language Translation. CVPR 2018.

[9] Yin, K., & Read, J. (2020). Better Sign Language Translation with STMC-Transformer. COLING 2020.

[10] Zhou, H., Zhou, W., Qi, W., Pu, J., & Li, H. (2021). Improving Sign Language Translation with Monolingual Data by Sign Back-Translation. CVPR 2021.

[11] Hao, A., Min, Y., & Chen, X. (2021). Self-Mutual Distillation Learning for Continuous Sign Language Recognition. ICCV 2021.

[12] Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.

[13] Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR, 21(140), 1-67.

[14] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI Blog.

[15] Liu, Y., et al. (2020). Multilingual Denoising Pre-training for Neural Machine Translation. TACL, 8, 726-742.

[16] Mokady, R., Hertz, A., & Bermano, A.H. (2021). ClipCap: CLIP Prefix for Image Captioning. arXiv:2111.09734.

[17] Li, J., Li, D., Xiong, C., & Hoi, S.C.H. (2022). BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. ICML 2022.

[18] Cao, Z., Hidalgo, G., Simon, T., Wei, S.E., & Sheikh, Y. (2019). OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields. IEEE TPAMI.

[19] Ko, S.K., Kim, C.J., Jung, H., & Cho, C. (2019). Neural Sign Language Translation based on Human Keypoint Estimation. Applied Sciences, 9(13).

[20] Tunga, A., Nuthalapati, S.V., & Wenger, J. (2021). Pose-based Sign Language Recognition using GNN and CNN. WACV 2021 Workshops.

[21] Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A Simple Framework for Contrastive Learning of Visual Representations. ICML 2020.

[22] He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. CVPR 2020.

[23] Duarte, A., Palaskar, S., Ventura, L., Ghadiyaram, D., DeHaan, K., Metze, F., Torres, J., & Giro-i-Nieto, X. (2021). How2Sign: A Large-scale Multimodal Dataset for Continuous American Sign Language. CVPR 2021.

[24] Graves, A., Fernandez, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML 2006.

[25] Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. WMT 2018.